

CSCI 8205 Milestone 1 Brief Report

Multi-GPU Pipeline Architecture for Real-Time Digital Inline Holographic Reconstruction

Students: Nicholas Bravo-Frank, Anlei Chen

Course: CSCI 8205: Design and Implementation of Multiprocessor Systems

Date: March 5, 2026 | **Period:** Weeks 1-4 (completed through Week 6)

***Note:** This document was formatted using Claude Sonnet 4. Full technical details available in complete milestone report.*

Key Achievements

Technical Accomplishments: We successfully implemented and validated a complete 6-stage holographic processing pipeline in C++ with native TensorRT integration, achieving performance improvements across all stages. The pipeline includes image loading, background subtraction, holographic reconstruction via 2D FFT, CNN-based particle detection, ML classification, and post-processing.

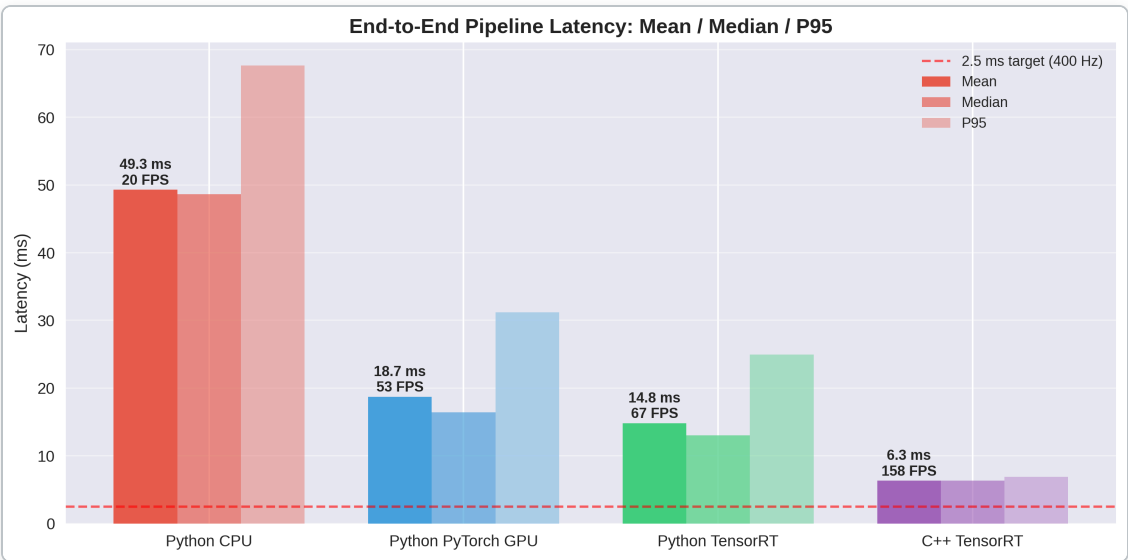


Figure 1: End-to-end performance: C++ TensorRT achieves 7.8× speedup with superior predictability

Performance Analysis: Our benchmarking revealed that the largest speedups come from eliminating Python/framework overhead rather than algorithmic improvements. The EMA stage shows a 38.2× improvement through kernel fusion, while the classification stage achieves 5.2× speedup by bypassing ultralytics overhead and using direct TensorRT calls.

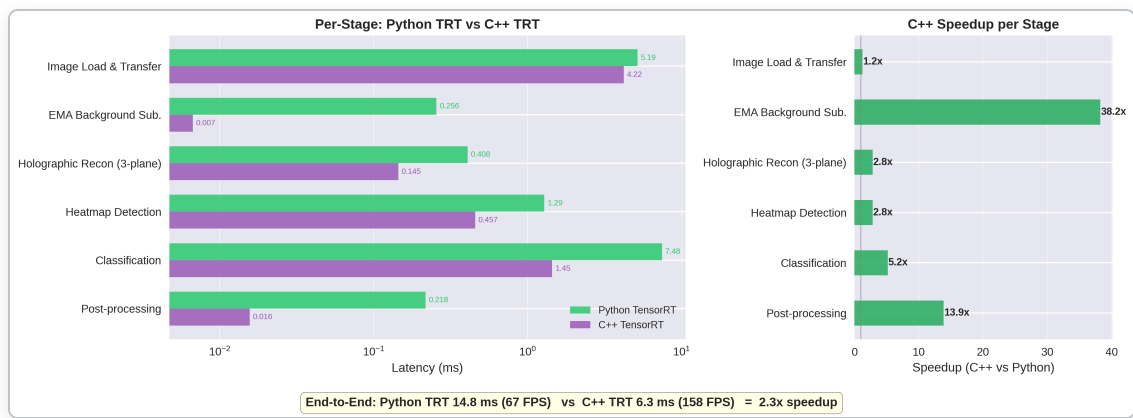


Figure 2: Per-stage performance comparison revealing framework overhead elimination

Performance Results

Implementation	Latency (ms)	FPS	Speedup	Predictability
Python CPU	49.3	20	1.0×	P95/P50 = 1.39×
Python TensorRT	14.8	67	3.3×	P95/P50 = 1.91×
C++ TensorRT	6.3	158	7.8×	P95/P50 = 1.08×

Architectural Insights: Extended roofline analysis revealed that TensorRT stages achieve only 1-2% of theoretical GPU peak performance, indicating significant potential for concurrent execution via CUDA streams. The pipeline transformed from compute-bound (Python) to I/O-bound (C++), with disk loading now consuming 67% of end-to-end latency.

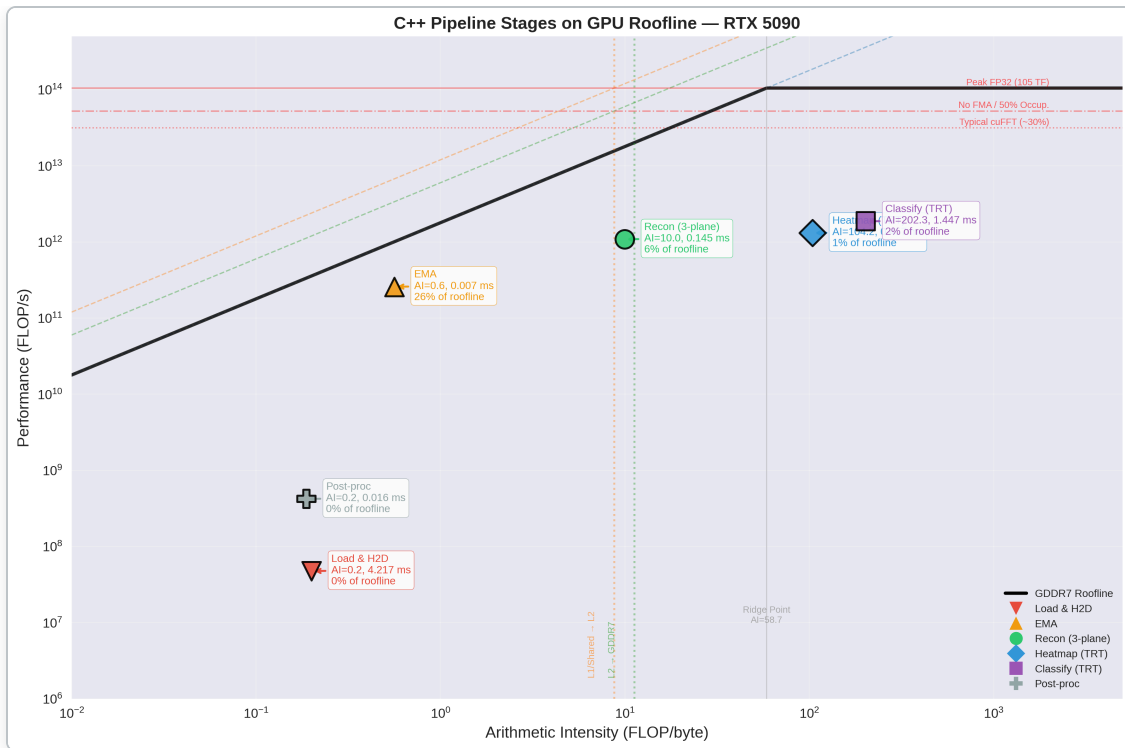


Figure 3: Pipeline stages on GPU roofline showing optimization opportunities

Schedule Progress

Ahead of Schedule: We completed our original Week 1-4 objectives by Week 2-3 and advanced into work planned for Weeks 5-8, including full pipeline implementation and comprehensive benchmarking. This positions us well for ambitious ring buffer and multi-GPU work in the remaining weeks.

Next Phase (Week 7-8): Ring buffer implementation for all 6 pipeline stages to eliminate I/O bottleneck and enable concurrent execution. Expected outcome: sub-2ms end-to-end latency (~500 FPS), exceeding the 400 Hz camera target.

Technical Impact

Research Contributions: This work provides a roofline analysis of our digital inline holographic processing pipelines, establishes cross-implementation validation protocols across Python/C++ and CPU/GPU architectures, and demonstrates the feasibility of real-time holographic particle detection at camera-limited framerates.

Performance Summary — Holographic Reconstruction							
Config	Py CPU (ms)	Py GPU (ms)	C++ OMP (ms)	C++ CUDA (ms)	Best OMP Thr	GPU vs Py CPU	CUDA vs OMP
256×256 × 1p	2.35	0.302	1.35	0.102	16	8×	13.2×
256×256 × 5p	5.08	0.304	3.39	0.116	32	17×	29.2×
256×256 × 20p	12.05	0.353	13.33	0.188	32	34×	70.9×
512×512 × 1p	2.90	0.277	4.00	0.229	32	11×	17.5×
512×512 × 5p	11.89	0.330	13.14	0.317	32	36×	41.4×
512×512 × 20p	49.93	0.536	43.29	0.914	32	93×	47.4×
1024×1024 × 1p	6.43	0.317	10.02	0.354	32	20×	28.3×
1024×1024 × 5p	51.69	0.593	36.91	1.026	32	87×	36.0×
1024×1024 × 20p	211.69	3.247	162.40	3.491	32	65×	46.5×

Figure 4: Comprehensive performance summary across all implementations and configurations

Conclusion: The 7.8× performance improvement demonstrates significant progress toward real-time holographic processing. The current pipeline implementation has achieved substantial optimization gains, with further improvements requiring parallelism strategies. Our next step involves implementing ring buffers and refined memory management for each of the 6 stages to push performance closer to hardware saturation. We are well positioned to meet the 400 Hz target and advance to multi-GPU scaling challenges in the project's final phase.